Third Eye

The Recording Part

What it is, why we need it, how to do it step by step, and everything we can build before a single run is recorded.

> **In one line:** recording means filming real people solving the jigsaw and playing Jenga in front of our cameras, some runs done correctly and some with mistakes planted on purpose, and then writing down (labelling) exactly what happened and when. That labelled footage is the only thing that can tell us whether Third Eye actually works on real video.

1. What we are doing in the recording part

Everything the project has been tested on so far is **scripted or synthetic**: the scenario files in scripts/scenarios/*.yaml describe a scene over time, and the jigsaw piece matcher has only been tested on generated images. The recording part replaces that with real footage. It has three pieces:

- **Record.** Film complete jigsaw runs with the overhead webcam over the ArUco board, and complete Jenga games with the iPhone Pro (colour + LiDAR depth) at one corner and a second camera at the opposite corner.
- **Plant mistakes.** In about half the runs the person makes one deliberate, pre-agreed mistake (an interior piece placed early, a piece rotated, a block pulled from the top layer, and so on). We know in advance what the mistake is, where, and roughly when.
- **Label.** Afterwards someone goes through each video and writes down the ground truth: when each piece or block moved, where it went, and which mistake (if any) happened at what time.

The result is two small datasets of our own, **THIRDEYE-JIGSAW** and **THIRDEYE-JENGA**: video files, a run sheet saying who did what, and a label file per run.

2. Why we are doing it

Reason	What it gives us
Real-world tuning	The piece matcher's thresholds (empty vs occupied cell, confidence gap) were set on synthetic images. Real lighting, shadows, glare, print quality and hands in the frame will move them. We need real frames to re-measure them.
Proving it works (evaluation)	Every result the project promises needs ground truth: precision and recall per error type, false alerts per correct run, delay from mistake to alert, Jenga slot-occupancy accuracy. Without labelled runs none of these numbers exist.
Why plant mistakes	Real people rarely make mistakes on cue. If we only filmed natural play we might see two errors all week. Planting them gives enough examples of each error type to measure recall, and because we planned them we already know the answer.
Why include correct runs	A system that alerts all the time catches every mistake but is useless. Correct runs measure false alarms, which is what a viewer of the live demo notices first.
Debugging offline	A recorded run can be played through the real pipeline again and again (--video run.mp4) after every code change, without setting up the rig. Bugs become reproducible.
Demo safety net	If something breaks on demo day (lighting, camera, laptop), a recorded run still shows the system working.

> **Note:** docs/Third_Eye_Project_Plan.md and the requirements still describe recording for the old LEGO plan (THIRDEYE-LEGO, 40-60 runs, 30 labelling hours). The jigsaw numbers below are my suggestion, adapted from those; the Jenga numbers come from the plan (DR-JEN-1 to 4). Agree them with the team and your mentor before starting.

3. The entire process, step by step

1 Decide what to record (the protocol)

One page the whole team agrees on before anyone touches a camera.

- **Jigsaw:** the 4x3 puzzle our spec covers (specs/jigsaw_4x3.json). Suggested: 30-40 runs from 3-4 people, about half with one planted mistake.
- **Jigsaw mistakes** (same as the existing scenarios): interior piece before border done (out of order); piece in the wrong cell; piece rotated; piece from another puzzle (foreign); finishing with a gap (missing).
- **Jenga:** 15-20 games from 3-4 people. Planted violations: pull from the top complete layer; start a new layer early; two-handed pull; put a block back in an illegal slot. Collapses happen naturally.
- Write a **run sheet** template: run ID, date, person, procedure, planted mistake (or 'none'), which piece/block, notes. Assign every mistake to a run in advance so each type gets roughly equal counts.

2 Consent and privacy

Required by NFR-PRIV-1 and DR-REL-2.

- Each person recorded signs a short consent form: one tick box for research use, a separate one for public release.
- Frame the workspace, not faces. Hands and arms only.
- Store the signed forms somewhere other than the video folder.

3 Build and fix the rig

The same setup for every run, so differences come from the person, not the camera.

- **Jigsaw:** webcam on a stand directly overhead; print the ArUco markers at 100% (scripts/make_markers.py) and tape them at the board corners; fix lamp position; matte background.
- **Jenga:** iPhone Pro on a tripod at one corner of the tower, streaming colour + depth through Record3D; second camera at the opposite corner; dark backdrop; tower on a marked spot so it is in the same place each game.
- Mark camera and tripod positions with tape so the rig can be rebuilt exactly after it is moved.
- Take the reference shots: a photo of the **empty board** (the matcher needs it) and a clean photo of the **box picture**.

4 Pilot run and check

Record one or two runs and push them through the whole pipeline before recording the rest.

- Check the files play, the frame rate is steady (aim 15-30 FPS) and the resolution is what you set.
- Check calibration: all four ArUco markers are found in every frame, even with a hand over the board.
- For Jenga, check depth is being saved, not just colour, and that both cameras cover the whole tower.
- Fix anything found now. A problem found after 30 runs means re-recording 30 runs.

5 Record the runs

The same routine every time.

- Fill in the run sheet row **before** pressing record (run ID, person, planted mistake).
- Start all cameras, then do a **sync mark** (a clap or a hand wave in view of both cameras) so the two Jenga views can be lined up later.
- Hold up a card with the run ID for a second at the start, so a video can never be confused with another.
- Do the run. For a planted mistake, make it look natural; do not pause or point at it.
- Stop recording, write the rough time the mistake happened on the run sheet, and name the file jigsaw_R012_personB.mp4 or similar.
- Record steadily across several days rather than all at the end; the old plan's lesson was to start early.

6 Store and back up

Raw footage is the one thing that cannot be regenerated.

- One folder per dataset: raw/ (videos, depth), runs.csv (the run sheet), labels/.
- Copy to a second place (team drive or cloud bucket) the same day. Do not commit videos to git.
- Never edit raw files; any trimming or conversion writes a new file.

7 Label the runs

Write down the ground truth for every run.

- **Jigsaw, per piece:** time it was placed, which piece, which cell, which rotation; plus the mistake type and time if one happened.
- **Jenga, per move:** move start/end time, source slot, destination slot, violation type (or none), and collapse time if the tower fell. The plan budgets about 8 person-hours for Jenga.
- Use CVAT or ELAN for time segments, or a simple CSV per run if that is faster. Pick one format and write it down.
- Have a second person check a few runs to catch labelling mistakes.
- Label a handful of frames with the true board / tower state, to measure state accuracy directly.

8 Split the data and freeze it

Required by NFR-REP-2.

- Split runs into a **tuning set** (used to set thresholds and debug) and a **test set** (only used for final numbers). Split by person so the test set has people the system was not tuned on.
- Write the split into a committed file and never change it after a result has been reported from it.

9 Use the recordings

Where the recording pays off.

- Tune the piece matcher on tuning-set frames (python -m thirdeye.perception.pieces).
- Run every recorded video through the live pipeline, save the session log, and compare its alerts against the labels.
- Report per error type: precision, recall, false alerts per correct run, median alert delay; for Jenga, slot occupancy accuracy and violation precision/recall.
- Pick the best correct and best mistake runs as backup demo videos.

4. What we can do now, without any recording

A lot of the remaining work does not need real footage: it can be built and tested on the scripted replays and text alone, so it is ready the moment recordings arrive. In rough priority order:

#	Work item	Why it doesn't need recording	Where
1	**Evaluation scripts**: precision/recall per error, false alerts per run, alert delay, latency	Score them against the scripted scenarios, whose right answers are already written in their expect: blocks. When real labels arrive, the same scripts just read them.	new eval/
2	**Label format** and a loader for it	Deciding the label format now means recording and labelling (steps 5 and 7) produce exactly what the scorer reads.	eval/, docs
3	**Compiler eval corpus**: 15-20 short text manuals and rulebooks with hand-written correct specs	Pure text. manuals/eval_corpus/ is currently empty. The plan calls this 'the one result that survives whether or not the vision works'.	manuals/eval_corpus/
4	**LLM compiler for jigsaw and Jenga**: jigsaw instructions to phases; Jenga rulebook to a ConstraintSpec	Text in, spec out, checked by the existing validator and by replaying the scenarios with the compiled spec.	compiler/
5	**Jigsaw live loop**: webcam to matcher to board tracker (like live.py for LEGO)	Can be built and tested with the webcam pointed at the board live, and with synthetic frames. Needs no dataset.	live.py / new
6	**Diagonal-corner Jenga lattice**, two cameras merged	Geometry: can be developed against synthetic views of a tower. lattice.py currently assumes two face-on side views.	state/jenga/
7	**Record3D input reader** (colour + depth from the iPhone)	Needs the phone, not a dataset. Also becomes the recording tool for step 5.	perception/
8	**LLM session report** from the session log	It reads only the JSON log, never video, so logs from replays are enough to build it.	explain/
9	**FastAPI + WebSocket server and web UI**: checklist, alerts, timeline	Drive it from thirdeye.replay exactly as a camera would.	server/, ui/
10	**Robustness tests**: add noise to replays (flicker, missed frames, long occlusion)	Measures false alarms and the 'wait instead of guessing' behaviour before real noise shows up.	tests/, scripts
11	**Step recognizer on a public dataset** (Assembly101 or EgoPER)	Uses someone else's recorded, labelled video. Check access and licence first (DR-PUB-1).	temporal/
12	**Update the plan and requirements docs** to jigsaw + Jenga	Both still describe the LEGO plan, including this recording plan.	docs/
13	**Recording tool**: a small script that saves synced video + depth with run IDs and fills the run sheet	Makes step 5 faster and less error-prone.	scripts/

Suggested order

- **This week:** items 1 and 2 (evaluation + label format) and 3 (compiler corpus). They lock in what 'working' means and give measurable results that do not depend on the cameras.
- **Next:** items 5 and 7 (jigsaw live loop, Record3D reader), which are also what the recording itself needs, then item 13, then start recording.
- **In parallel, whoever is free:** 4, 8, 9 and 12.

> **A few things on the list do need the camera**, just not a recorded dataset: the jigsaw live loop and the Record3D reader need the webcam and the iPhone plugged in. Tuning the matcher's thresholds for real (the last jigsaw roadmap item before the live loop) is the one thing that really does need real photos. A few still photos of the board are enough to start; you don't need the full recording for that.